j4_stepper_eyes

Eye-pop stepper driver board for the Johnny 4 robot, and the last node of the stepper daisy chain. Runs on an ESP32-DEV (WROOM) module. It drives the two eye-pop steppers through TMC2209 drivers and reports their health back upstream. Step pulses run on the ESP32 RMT peripheral via FastAccelStepper.

This board was briefly retired (its two channels were consolidated onto the neck board), then revived as of v0_6: with its own ESP32 there are enough pins to give **every limit switch its own input**, so the firmware knows exactly which endpoint was hit.

Daisy chain

```
j4_receiver (TTGO)
      |  UART1
      v
j4_stepper_neck   (3x DM556TE)
      |  UART2
      v
j4_stepper_eyes   (2x TMC2209)   <-- this board
```

This board is the end of the chain. It receives the eye-pop value forwarded by j4_stepper_neck and sends its status back up the same link.

What this board does

- Receives an eye-pop target (`<eye>\n` , 0-3200) from j4_stepper_neck over UART1
- Drives 2 eye-pop steppers via TMC2209 drivers, **each on its own STEP/DIR pins** (two independent FastAccelStepper channels) but **commanded to identical targets** so the eyes pop together
- Polls each TMC2209's DRV_STATUS for faults (overtemp prewarning/shutdown, short, open-load) and reports a status string back upstream
- Filters out tiny commanded changes (threshold) so the motors do not jitter at rest

Why two independent channels instead of one shared signal

The eyes always pop to the same value, so a single shared STEP/DIR signal would also keep them in sync. They are kept on **separate channels** so each eye can be **trimmed or homed independently** later (re-zeroing one motor after a skipped step, or sensorless homing per driver via StallGuard) without disturbing the other. The two TMC2209s sit on a shared UART bus at separate addresses (0b00 / 0b01), so each is individually addressable for that.

Hardware

Component	Details
Microcontroller	ESP32-DEV (WROOM-32)
Eye-pop drivers	2x TMC2209 (shared single-wire UART, addresses 0b00 / 0b01)

Link protocol (UART1, to/from j4_stepper_neck)

Direction	Message	Meaning

in	`<eye>\n`	eye-pop target (0-3200, same scale as the neck)
in	`CAL\n`	run a homing/calibration pass on both eyes
out	`S:<text>\n`	status: `S:OK`, `S:EL OT`, `S:EL MIN`, `S:ER MAX`, etc.

The status tokens are `EL` / `ER` (eye left/right) plus `OT` (overtemp shutdown), `SHORT`, `OTPW` (overtemp prewarning), `OPEN` (open load), or `MIN` / `MAX` (that exact limit switch is engaged or its wire is broken). The parser uses a fixed char buffer (no Arduino `String`).

Pin assignments

GPIO	Function
26 / 25	eye-left STEP / DIR
33 / 32	eye-right STEP / DIR
17 / 16	TMC2209 UART TX / RX (shared PDN_UART bus)
4 / 13	Upstream UART1 RX / TX (to neck board)
34	eye-left MIN limit (input-only, external 10k pull-up)
39	eye-left MAX limit (input-only, external 10k pull-up)
36	eye-right MIN limit (input-only, external 10k pull-up)
35	eye-right MAX limit (input-only, external 10k pull-up)

The links and step/dir use plain bidirectional GPIOs (strapping pins 0/2/12/15 and flash pins 6-11 are avoided). The four limit inputs use the input-only pins 34/35/36/39, each with an external 10k pull-up to 3.3V (input-only pins have no internal pull-up). This board has no battery-sense pin, so GPIO 35 is safe to use.

Limit switches are not required for normal operation, so you can wire them whenever you add them.

Pin diagram

Same board as j4_stepper_neck (ESP32-D0WD, LOLIN32-Lite style). Physical pin order, component side up, USB-C at the BOTTOM; rails read top-to-bottom.

```
     LEFT rail                              RIGHT rail
                          +--------------+
  limit ER MIN -- 36 |    |              |    | 3V3   3.3V (limit pull-ups)
  limit EL MAX -- 39 |    |              |    | 22    (free / onboard LED)
         reset -- EN |    |  ESP32-D0WD  |    | 19    (free)
  limit EL MIN -- 34 |    |  (LOLIN32-   |    | 23    (free)
  limit ER MAX -- 35 |    |   Lite style)|    | 18    (free)
     DIR eye-R -- 32 |    |              |    | 5     (free)
    STEP eye-R -- 33 |    |              |    | 17    TMC2209 UART TX
     DIR eye-L -- 25 |    |              |    | 16    TMC2209 UART RX
    STEP eye-L -- 26 |    |              |    | 4     UART1 RX <- neck 22
        (free) -- 27 |    |              |    | 0     (free / BOOT)
        (free) -- 14 |    |              |    | 2     (free)
        (free) -- 12 |    |              |    | 15    (free)
```

```
        ground -- GND|   |              |   | 13    UART1 TX -> neck 35
                         +---[ USB-C ]--+

  Both TMC2209 share the UART bus (17/16); addresses 0b00 (eye-L) / 0b01 (eye-R).
  Address straps: eye-L MS1 -> GND, MS2 -> GND ; eye-R MS1 -> VIO, MS2 -> GND.
  GPIO 16 (RX) sits on the PDN_UART line; GPIO 17 (TX) reaches it through 1k.
  Limits: one NC switch per endpoint, each with a 10k pull-up to 3.3V
  (input-only pins have no internal pull-up).
```

Power and enable

VIO on each driver goes to 3.3V, VM to the 12-24V motor supply, and all grounds are common with the ESP32. Each driver's EN pin is tied to GND, so the drivers are always enabled.

Motion settings

Eye-pop motion matches the neck so it has the same smooth ramp: `EYE_SPEED_HZ` (3200), `EYE_ACCEL` (16000), `EYE_MICROSTEPS` (16), and `EYE_THRESHOLD` (20) mirror the neck values, and the eye value maps through `eye / 2` exactly like a neck axis. Only `EYE_CURRENT` is separate, since it sets the TMC2209 driver current for the small eye-pop motors (not a motion setting) and the neck's 2.1A would be far too much. Tune `EYE_CURRENT` to your motors.

Limit switches and calibration

Each eye has **one normally-closed (NC) switch per endpoint** -- four inputs total. Each switch shorts its input to GND while the eye is in range: LOW = in range, HIGH = that endpoint was hit **or** its wire broke (fail-safe). `MIN` is the home end that `CAL` drives toward; `MAX` is the far end. All four inputs are on input-only pins and need an **external 10k pull-up to 3.3V** each.

- **In normal operation** the safety check is direction-aware: an eye is stopped only when it is moving *toward* a tripped endpoint, so a stopped eye can always drive itself back off a limit.
- **On a `CAL` command** (sent down the chain by the neck board), each eye homes **independently** toward its MIN switch. An eye already sitting on the switch is detected immediately (the switch reads pressed) and just backs off and zeros. StallGuard (read over the TMC2209 UART, no extra pins) stays as a backup: it catches an eye that **jams short of the switch**. Hitting the MAX switch during homing stops the eye (wrong direction or a wiring fault).

The StallGuard threshold (`SG_SGTHRS` , `SG_STALL_LEVEL`) and the homing speed/back-off are `#define`s at the top of `main.cpp` and **need tuning on the real mechanism**.

Building and uploading

```
# Build
pio run

# Upload
pio run --target upload

# Monitor serial output
pio device monitor
```

Related projects

- **j4_stepper_neck** -- the neck stepper board upstream of this one, which forwards the eye value and relays this board's status